



National University of Modern Languages, Islamabad

LAB 7

Submitted To:	Sir Shahid
Submitted By:	Tayyaba Ejaz
Roll Number:	2751
Subject:	MAD
Due Date:	12th December 2025
Semester:	8th
Course:	BSCS, Morning

Faculty of Engineering and Computer Science

Table of contents

1. Lab Task for Attendance App	3
1.1 Description	3
1.2 Code	3
1.2.1 datasource: app_preferences_local_data_source.dart	3
1.2.2 datasource: app_preferences_local_data_source_impl.dart.....	4

1. Lab Task for Attendance App

A Flutter app for a university is introducing a “Quick Access Mode” feature. When Quick Access Mode is enabled, the app skips the welcome tips screen on the next launch and takes the user directly to the main dashboard. This preference must be saved locally so it still works even after the app is closed and reopened. Design the Data Layer structure to manage this locally stored preference:

1. Create an abstract class/interface named `AppPreferencesLocalDataSource`.
2. Implement a concrete class named `AppPreferencesLocalDataSourceImpl` using the `shared_preferences` package.
3. Store and retrieve a boolean value using the key: `'quick_access_enabled'`.
4. Your implementation must include:
 - o A method to enable Quick Access Mode (save true)
 - o A method to disable Quick Access Mode (save false)
 - o A method to read whether Quick Access Mode is enabled (return false by default if nothing is stored yet)

Write the complete code for the concrete implementation class including the required methods.

1.1 Description

This task focuses on designing the **data layer** to manage the **Quick Access Mode** feature in a Flutter application. The user’s preference is stored locally using the **SharedPreferences** package to ensure persistence even after the application is closed and reopened. An abstract local data source defines the contract for storing and retrieving the preference, while a concrete implementation handles the actual read and write operations using a boolean value. This design promotes clean architecture by separating data handling from UI and business logic, ensuring scalability and maintainability.

1.2 Code

1.2.1 datasource: `app_preferences_local_data_source.dart`

```
abstract class AppPreferencesLocalDataSource {  
  
    Future<void> enableQuickAccess();  
  
    Future<void> disableQuickAccess();  
  
    Future<bool> isQuickAccessEnabled();  
}
```

```
}
```

1.2.2 datasource: app_preferences_local_data_source_impl.dart
import 'package:shared_preferences/shared_preferences.dart';
import 'app_preferences_local_data_source.dart';

```
class AppPreferencesLocalDataSourceImpl  
    implements AppPreferencesLocalDataSource {  
  
    static const String _quickAccessKey = 'quick_access_enabled';  
  
    @override  
    Future<void> enableQuickAccess() async {  
        final prefs = await SharedPreferences.getInstance();  
        await prefs.setBool(_quickAccessKey, true);  
    }  
  
    @override  
    Future<void> disableQuickAccess() async {  
        final prefs = await SharedPreferences.getInstance();  
        await prefs.setBool(_quickAccessKey, false);  
    }  
  
    @override  
    Future<bool> isQuickAccessEnabled() async {  
        final prefs = await SharedPreferences.getInstance();  
        return prefs.getBool(_quickAccessKey) ?? false;  
    }  
}
```
